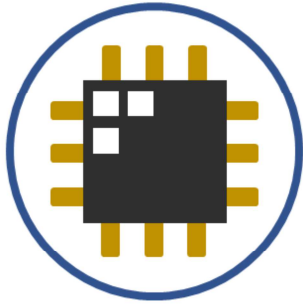
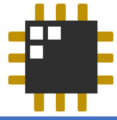




EEE 143: Switching Theory and Digital Logic Design

Lecture 12: Synthesis review, One-hot encoding

1

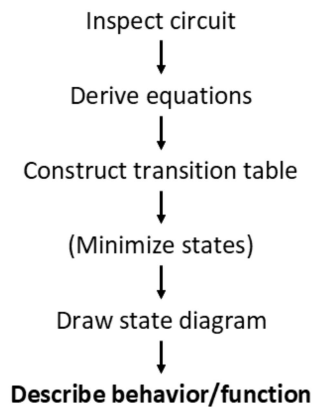


Last lecture: Analysis vs Synthesis

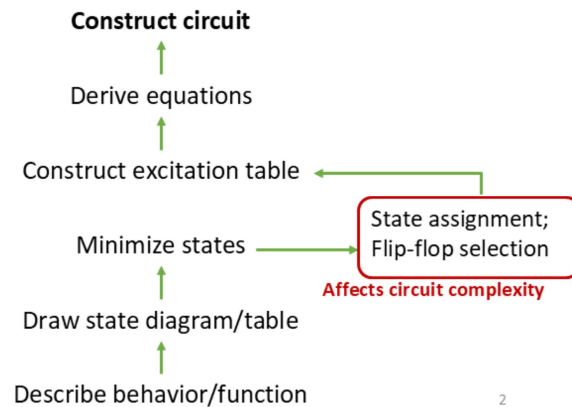
From circuit to state diagram...

From state diagram to circuit

Analysis



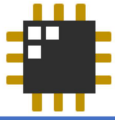
Synthesis



2

Before we start today's lesson, let's quickly review two things we already learned in past lectures. First, we learned how to analyze sequential circuits step by step. This means looking at the circuit to see what parts it has—like inputs, outputs, and flip-flops—and how they are connected. Then, we wrote the input and output equations using logic rules. After that, we made a transition table to show how the circuit moves from one state to another. We also made the circuit simpler by combining states that behave the same. Finally, we drew a state diagram to show these transitions and looked at how the outputs change based on the inputs and states.

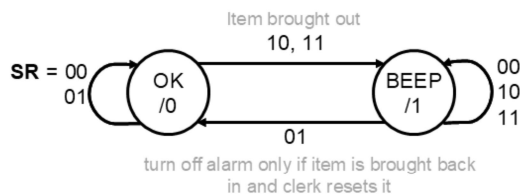
Now that we've already learned how to analyze a circuit, we'll move on to something new: designing a circuit from scratch. This time, we start with a description of what the circuit should do. From there, we create a state table and a state diagram to show how it should behave. Then, we simplify the states if needed, make an excitation table to figure out what the flip-flops need to do, write the logic equations, and finally, build the actual circuit.



Last lecture: Translating word problems

Synthesis: from word problem (state diagram) to circuit

- Given a *desired* functionality, create a circuit that *behaves* as desired



Present State	Next State				A
	SR = 00	01	10	11	
0	0	0	1	1	0
1	1	0	1	1	1

As long as we drew the correct state diagram, *everything else is methodical!*

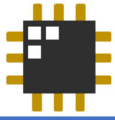
Translating a word problem into a state diagram correctly is critical

Let's take a moment to recall an important idea we've already touched on:

synthesis, or the process of designing a circuit from a given word problem or desired behavior.

When we're given a description of what a circuit should do, our goal is to build a circuit that behaves exactly that way. The key step here is creating the correct **state diagram**. If we can draw the state diagram properly, everything that follows—like making the state table, excitation table, writing equations, and building the circuit—is just a step-by-step process.

So remember: **translating the word problem into a correct state diagram is the most important part**. Once that's done right, the rest is mostly mechanical.



Last lecture: Completing the excitation table

Inputs		Present State	Next State	Excitation	Output
S	R	Q^n	Q^{n+1}	D	A
0	0	0			
0	0	1			
0	1	0			
0	1	1			
1	0	0			
1	0	1			
1	1	0			
1	1	1			

All combinations



What should be next state and output be?

Present State	Next State				A
	SR = 00	01	10	11	
0	0	0	1	1	0
1	1	0	1	1	1

Let's have another recall specifically about how we complete the excitation table.

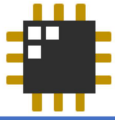
We were working with a system that had two inputs, **S** and **R**, and two states: **0 (OK)** and **1 (BEEP)**. After analyzing the system behavior and drawing the state table, we moved on to the excitation table. This table helped us figure out what logic we needed to apply to the flip-flop input, depending on the **current state** and the **inputs**.

We looked at all possible combinations of S and R with the present state, and determined the **next state** and **output** based on the behavior we already defined. Let's briefly go through what we saw:

- When **SR = 00** and the current state is 0, the system stayed in state 0 with an output of 0.
- For **SR = 00** and present state 1, it stayed in state 1 with an output of 1.
- **SR = 01** with state 0 stayed at 0, output 0.
- **SR = 01** with state 1 went back to 0 (because of the reset), output was 1.
- **SR = 10** with state 0 moved to 1—indicating shoplifting detection—and output stayed 0 during the transition.
- **SR = 10** with state 1 stayed in state 1, output 1.

- **SR = 11** with state 0 transitioned to state 1, output remained 0.
- And **SR = 11** with state 1 stayed in state 1, output 1.

This example showed how the excitation table gives us a clear map of how the system behaves and what logic is needed for each case.



Last lecture: Completing the excitation table

Inputs		Present State	Next State	Excitation	Output
S	R	Q^n	Q^{n+1}	D	A
0	0	0	0		0
0	0	1	1		1
0	1	0	0		0
0	1	1	0		1
1	0	0	1		0
1	0	1	1		1
1	1	0	1		0
1	1	1	1		1

All combinations

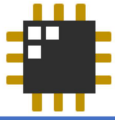


What should the flipflop input be to reach the desired next state?

Present State	Next State				A
	SR = 00	01	10	11	
0	0	0	1	1	0
1	1	0	1	1	1

Now, moving on to the determining the excitation for the flip-flop. We used a **D flip-flop**, which made things simple. Remember, the D flip-flop just passes its input directly to the output on the clock edge—so whatever we put into D becomes the next state.

Because of this, when we were building the excitation table, all we had to do was **take the values from the next state column**—which we had already figured out—and **copy them into the D input column**. That's it! The D input is simply equal to the next state for each combination of inputs and current state.



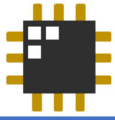
Last lecture: Completing the excitation table

Inputs		Present State	Next State	Excitation	Output
S	R	Q^n	Q^{n+1}	D	A
0	0	0	0	0	0
0	0	1	1	1	1
0	1	0	0	0	0
0	1	1	0	0	1
1	0	0	1	1	0
1	0	1	1	1	1
1	1	0	1	1	0
1	1	1	1	1	1

└──────────┘
All combinations

Present State	Next State				A
	SR = 00	01	10	11	
0	0	0	1	1	0
1	1	0	1	1	1

Now, we've completed the excitation table.



Last lecture: Boolean expression derivation

Inputs		Present State	Next State	Excitation	Output
S	R	Q^n	Q^{n+1}	D	A
0	0	0	0	0	0
0	0	1	1	1	1
0	1	0	0	0	0
0	1	1	0	0	1
1	0	0	1	1	0
1	0	1	1	1	1
1	1	0	1	1	0
1	1	1	1	1	1

All combinations

Present State	Next State				A
	SR = 00	01	10	11	
0	0	0	1	1	0
1	1	0	1	1	1

Using K-maps to derive the Boolean expressions

SR	Q			
	00	01	11	10
0	0	0	1	1
1	1	0	1	1

$$D = S + QR'$$

SR	Q			
	00	01	11	10
0	0	0	0	0
1	1	1	1	1

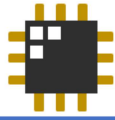
$$A = Q$$

Next, deriving the next state and output equations. Once we had our excitation table completed, the next step was to **simplify the logic expressions** for both the flip-flop input and the output—and for that, we used **Karnaugh maps (K-maps)**.

Remember, the goal with K-maps is to make the logic expression as simple as possible by grouping together adjacent 1s in the map—groups of 1, 2, 4, or 8—so we can write shorter and cleaner Boolean equations.

We referred to the K-maps shown on the bottom left to guide our simplification. After grouping and simplifying, we ended up with the following:

- **$D = S + QR'$** → This tells us what the input to the D flip-flop should be, which then determines the **next state** of the system.
- **$A = Q$** → This tells us the **output**, which means the alarm is on ($A = 1$) when the system is in the **BEEP** state ($Q = 1$).



Last lecture: Using a different flipflop

Inputs		Present State	Next State	Excitation		Output
S	R	Q^n	Q^{n+1}	J	K	A
0	0	0	0	0	x	0
0	0	1	1	x	0	1
0	1	0	0	0	x	0
0	1	1	0	x	1	1
1	0	0	1	1	x	0
1	0	1	1	x	0	1
1	1	0	1	1	x	0
1	1	1	1	x	0	1

A = Q still, while the FF inputs are:

SR				
	00	01	11	10
Q				
0	0	0	1	1
1	x	x	x	x

$$J = S$$

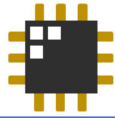
SR				
	00	01	11	10
Q				
0	x	x	x	x
1	0	1	0	0

$$K = S'R$$

Then, we tried doing it all again using a JK flipflop.

After filling out the excitation table and using Karnaugh maps to simplify, we came up with these expressions:

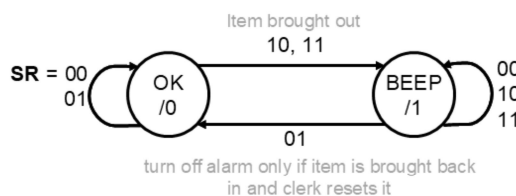
- $J = S$
- $K = S'R$



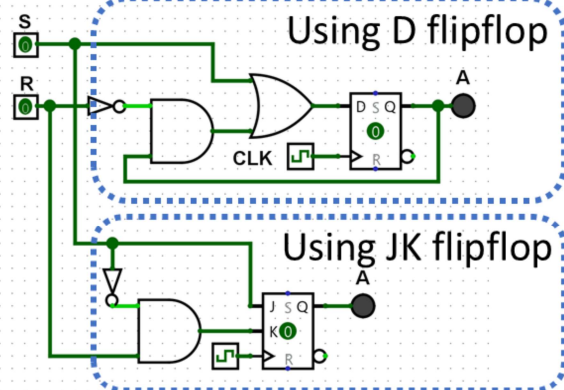
Last lecture: Synthesis summary

Synthesis: from word problem (state diagram) to circuit

- Given a *desired* functionality, create a circuit that *behaves* as desired



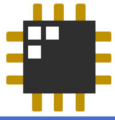
Present State	Next State				A
	SR = 00	01	10	11	
0	0	0	1	1	0
1	1	0	1	1	1



Flipflop selection matters!

What's interesting here is that this version, using the JK flip-flop, ended up needing **one less OR gate** compared to the version with the D flip-flop.

This reminds us of an important point: **the type of flip-flop we choose can affect how simple or complex our final circuit is.** So when designing a circuit, it's helpful to think about which flip-flop might give us the most efficient result.



Motivating another form of state encoding

The classical method of state assignment attempts to optimize the number of flipflops for the circuit implementation

- We can represent at most 2^n states using n flipflops
- ... but we have to follow some 'guidelines'
 - 1-bit change → fewer flipflop transitions → simpler equations → simpler circuit!
- ... also, flipflop choice matters.

Trying to optimize a 'dense' state assignment to create the simplest circuit is hard!

10

Let's now recall the **classical method of state assignment**, which is all about optimizing how we use flip-flops to implement our circuit.

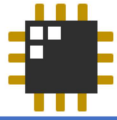
The basic idea is that with **n flip-flops**, we can represent **up to 2^n states**. So, if we have, say, 3 flip-flops, we can represent up to 8 states.

However, when we do state assignment, we need to follow some important **guidelines** to keep the circuit as simple as possible. Here are the key points to remember:

- 1. 1-bit change:** We want our states to change by only 1 bit at a time when possible. This reduces the number of flip-flop transitions between states.
- 2. Fewer flip-flop transitions:** The fewer transitions we have between states, the simpler the logic will be.
- 3. Simpler equations:** When the state transitions are simpler, the Boolean equations we derive will be easier to work with.
- 4. Simpler circuit:** All of this leads to a more efficient and less complex circuit overall!

Also, don't forget that **flip-flop choice matters**.

It's not always easy to optimize a "dense" state assignment (one with lots of states) to create the simplest circuit. Some flip-flops may work better for certain assignments, so picking the right one is key.



What if...

Instead of trying to '*optimize*' a '*dense*' assignment, **simply assign one flipflop to correspond to one state**

- We'll use D flipflops for this implementation
- We would end up needing n D flipflops to represent n states
- At any given time (clock cycle):
 - only 1 flipflop is **ON** (corresponding to the current state you are in)
 - the other flipflops are **OFF** (since you are not in that state)

This is called 'one-hot' encoding

11

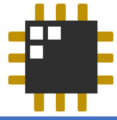
Now, let's move on to a different approach for state assignment called **one-hot encoding**.

Instead of trying to optimize a "dense" state assignment like we did before, in one-hot encoding, we **assign one flip-flop to each state**. This means for n states, we need n flip-flops. For this implementation, we'll use **D flip-flops**.

Here's how it works:

- At **any given time** (or clock cycle), **only one flip-flop is ON**, corresponding to the current state we are in.
- The **other flip-flops are OFF**, because we are not in those states at that moment.

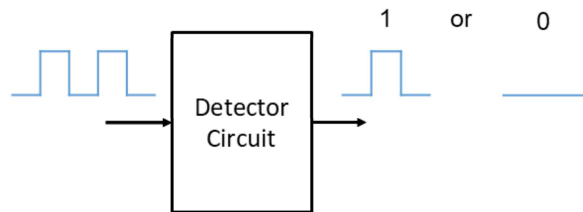
So, each flip-flop "represents" a state, and only the flip-flop corresponding to the active state is set to 1. This method is called **one-hot encoding** because, at any time, you have a "hot" (active) flip-flop representing the current state, while the others remain "cold" (inactive).



Example

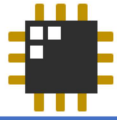
Beginning of message detector:

Design a sequential logic circuit that outputs a 1 (permanently) when three consecutive ones are present in a digital line. Use positive edge-triggered D flip-flops. Assume a Mealy machine behavior



12

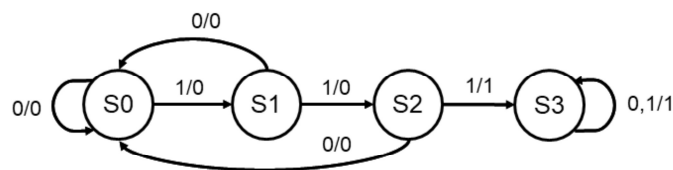
Let's now apply **one-hot encoding** to the example we discussed in the last lecture. In this case, we need to design a sequential logic circuit that outputs **1** permanently once it detects three consecutive 1s on a digital input line. To implement this, we'll use **positive edge-triggered D flip-flops**.



Example

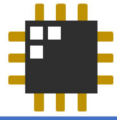
Beginning of message detector:

Design a sequential logic circuit that outputs a 1 (permanently) when three consecutive ones are present in a digital line. Use positive edge-triggered D flip-flops. Assume a Mealy machine behavior

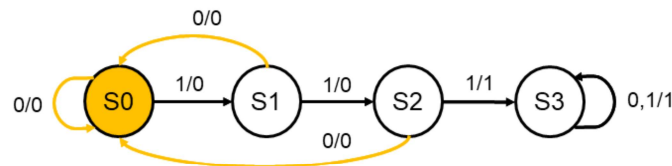


13

Using the **one-hot encoding** method, we arrive at the updated **state diagram** shown below.



Example



We can obtain the next state and output equations directly from the diagram!

The next state is **S0** if you're in [state **S0** AND **X=0**] OR in [state **S1** AND **X=0**] OR in [state **S2** AND **X=0**]...

$$D_0 = Q_0X' + Q_1X' + Q_2X'$$

14

Let's now figure out the **next state equation for S0**. In one-hot encoding, remember that each state has its own flip-flop. So here, we're determining when the flip-flop for **S0** (let's call it **D0**) should be set to 1 in the next clock cycle.

To stay or return to **S0**, we look at **which situations bring us there**:

- If we're currently in **S0** and the input **X** is **0**, we stay in **S0**.
- If we're in **S1** and **X** is **0**, we go back to **S0**.
- If we're in **S2** and **X** is **0**, we also return to **S0**.

So, in terms of logic:

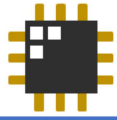
- We want **D0 = 1** when any of those conditions are true.
- That gives us the equation:

$$D0 = Q0 \cdot X' + Q1 \cdot X' + Q2 \cdot X'$$

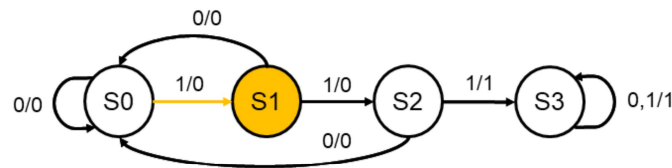
Here, **Q0**, **Q1**, and **Q2** represent the current state (which flip-flop is active), and **X'** means the input **X** is **0**.

So, the flip-flop for **S0** turns on in the next cycle if the system is currently in **S0**,

S1, or S2, and X is 0. This is how we build the next-state logic in a one-hot encoded design.



Example



We can obtain the next state and output equations directly from the diagram!

The next state is **S1** if you're in [state **S0** **AND** **X=1**]...

$$D_1 = Q_0 X$$

15

Let's continue by looking at how we determine the **next-state equation for S1** in our one-hot encoded design.

To move into **state S1**, we need to think about **when that transition happens**.

In this case, the only way to get to **S1** is if:

- We are currently in **S0**, and the input **X** is **1**.

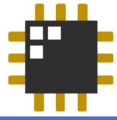
In terms of logic, this gives us:

$$D_1 = Q_0 \cdot X$$

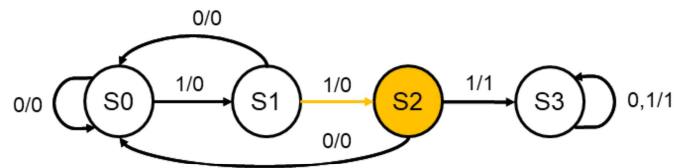
Here:

- **Q0** is the flip-flop for S0 (meaning we're in state S0),
- **X** is the input, and
- **D1** is the input to the flip-flop for S1.

So this expression means the flip-flop for **S1** will be activated (set to 1) on the next clock cycle only in that one specific condition: when we're in **S0** and receive a **1** on the input line.



Example



We can obtain the next state and output equations directly from the diagram!

The next state is **S2** if you're in [state **S1** **AND** **X=1**]...

$$D_2 = Q_1 X$$

16

Let's now look at how we get the **next-state equation for S2**.

The system goes to **S2** only when:

- It is currently in **S1**, and the input **X** is **1**.

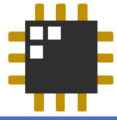
This gives us the equation:

$$\mathbf{D2 = Q1 \cdot X}$$

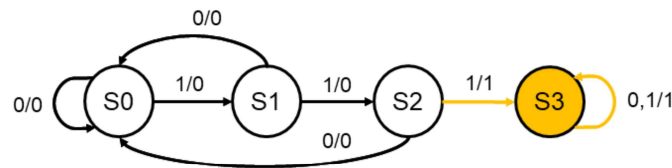
Here:

- **Q1** represents that we're currently in **S1**,
- **X** is the input, and
- **D2** is the input to the flip-flop for **S2**.

So, the flip-flop for **S2** will turn ON in the next clock cycle only when the system is in **S1** and receives a **1** on the input line.



Example



We can obtain the next state and output equations directly from the diagram!

The next state is **S3** if you're in [state **S2** **AND** **X=1**] **OR** in [state **S3**]...

$$D_3 = Q_2X + Q_3$$

17

Let's finish up the next state equations by looking at how we get the **next-state equation for S3**.

We move into **state S3** in two situations:

1. If we are currently in **S2** and the input **X** is **1** — this means we've detected the third consecutive 1, so we transition to S3.
2. If we are already in **S3** — we stay in this state permanently once it's reached.

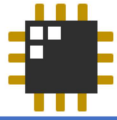
This leads to the equation:

$$D_3 = Q_2 \cdot X + Q_3$$

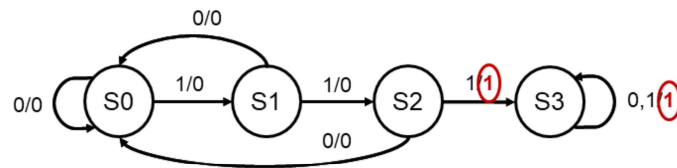
Where:

- **Q2** means we're in **S2**,
- **X** is the input,
- **Q3** means we're already in **S3**, and
- **D3** is the input to the flip-flop for **S3**.

This setup ensures that once we detect three 1s in a row, the system enters **S3**, and from there, it stays in **S3** no matter what.



Example



We can obtain the next state and output equations directly from the diagram!

The output is 1 if you're in [state **S2** **AND** **X=1**] **OR** in [state **S3**]...

$$Z = Q_2X + Q_3$$

18

Now let's determine when the **output Z** should be **1**.

In this design, we want the output to turn on **only when we've detected three consecutive 1s**. That happens:

1. When we are in **state S2** and the input **X is 1** — this means we've just received the third 1 in a row.
2. When we are already in **state S3** — the system has already detected three 1s and should continue to output 1.

This gives us the equation:

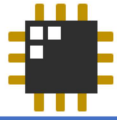
$$Z = Q_2 \cdot X + Q_3$$

Where:

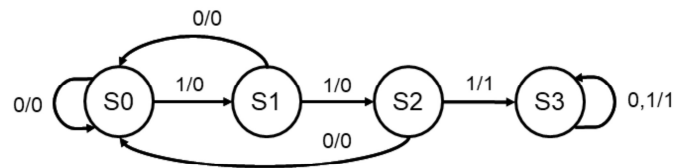
- **Q2** is active when the system is in **state S2**,
- **X** is the input,
- **Q3** is active when the system is in **state S3**, and
- **Z** is the output.

This logic ensures that the output turns on exactly when it should—and stays on

once the condition has been met.



Example



Next state equations:

$$D_0 = Q_0X' + Q_1X' + Q_2X'$$

$$D_1 = Q_0X$$

$$D_2 = Q_1X$$

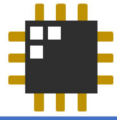
$$D_3 = Q_2X + Q_3$$

Output equation:

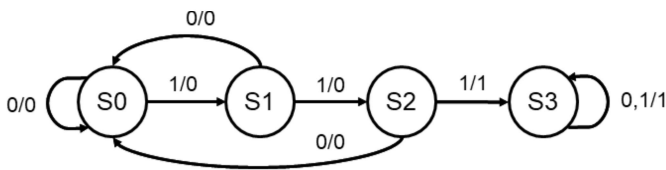
$$Z = Q_2X + Q_3$$

19

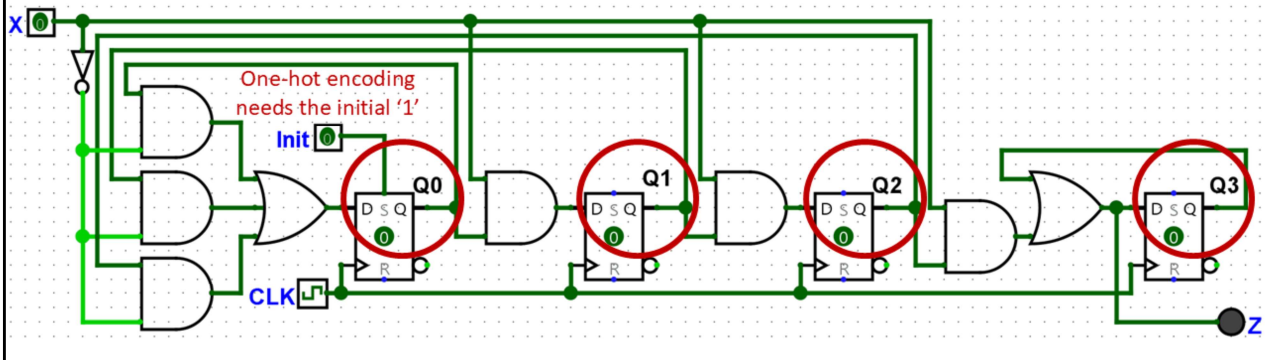
Here's a summary of the **next-state** and **output equations** for this example.



Example



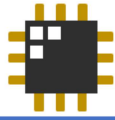
Pay attention on how each flipflop turns ON one at a time corresponding to the state you're currently on.



Let's now look at the **circuit simulation** based on the equations we just derived. The GIF on the slide shows how the actual circuit behaves as the input changes over time.

As you watch the animation, **pay close attention to the flip-flops**. Notice how **only one flip-flop is ON at any given moment** — this reflects the key idea behind **one-hot encoding**: each state has its own flip-flop, and only the flip-flop for the **current state** is active (i.e., outputting 1).

You can also see how the **output Z** matches exactly what we expected from the **state diagram** — it turns ON only after three consecutive 1s are detected and stays ON afterward. This confirms that our circuit logic and equations correctly implement the desired behavior.



Putting it into perspective...

Example 1

Beginning of message detection:
Design a sequential logic circuit that outputs a 1 (permanently) when three consecutive ones are present in a digital line. Use positive edge-triggered D flip-flops.

1.2 Draw the state diagram/table

1.3 Minimize states

1.4 Assign state variables

1.5 Choose flip-flops

1.6 Construct excitation table

1.6 Construct excitation table

1.6 Construct excitation table

1.6 Construct excitation table

1.6 Construct excitation table

1.6 Construct excitation table

1.7 Derive next-state and output functions

1.7 Derive next-state and output functions

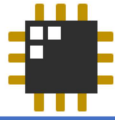
1.7 Derive next-state and output functions

1.9 (Bonus) Simulate and verify!

This is the classical way (previous lecture)

21

This slide shows the number of steps we took using the **classical approach**. You can see that we needed around **15 slides** to go from the word problem, through the state table, excitation table, Karnaugh maps, and finally the circuit. This method is thorough, but it can get **quite lengthy** and **complex**, especially as the number of states increases.



Putting it into perspective...

Example

Beginning of message detector:
Design a sequential logic circuit that outputs a 1 (permanently) when three consecutive ones are present in a digital line. Use positive edge-triggered D flip-flops.



Example

Beginning of message detector:
Design a sequential logic circuit that outputs a 1 (permanently) when three consecutive ones are present in a digital line. Use positive edge-triggered D flip-flops.



Example

We can obtain the next state and output equations directly from the diagram!

The next state is 0 if you're in [state S0 AND X=0] OR in [state S1 AND X=0] OR in [state S2 AND X=0].
 $D_0 = Q_0X' + Q_1X' + Q_2X'$



Example

Next state equations:
 $D_0 = Q_0X' + Q_1X' + Q_2X'$
 $D_1 = Q_0X$
 $D_2 = Q_1X$
 $D_3 = Q_2X + Q_1$

Output equation:
 $Z = Q_2X + Q_2$
The output is 1 if you're in [state S2 AND X=0] OR in [state S2].



Example

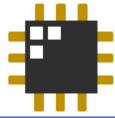
Pay attention on how each flip-flop turns OFF once at a time corresponding to the state you're currently in.



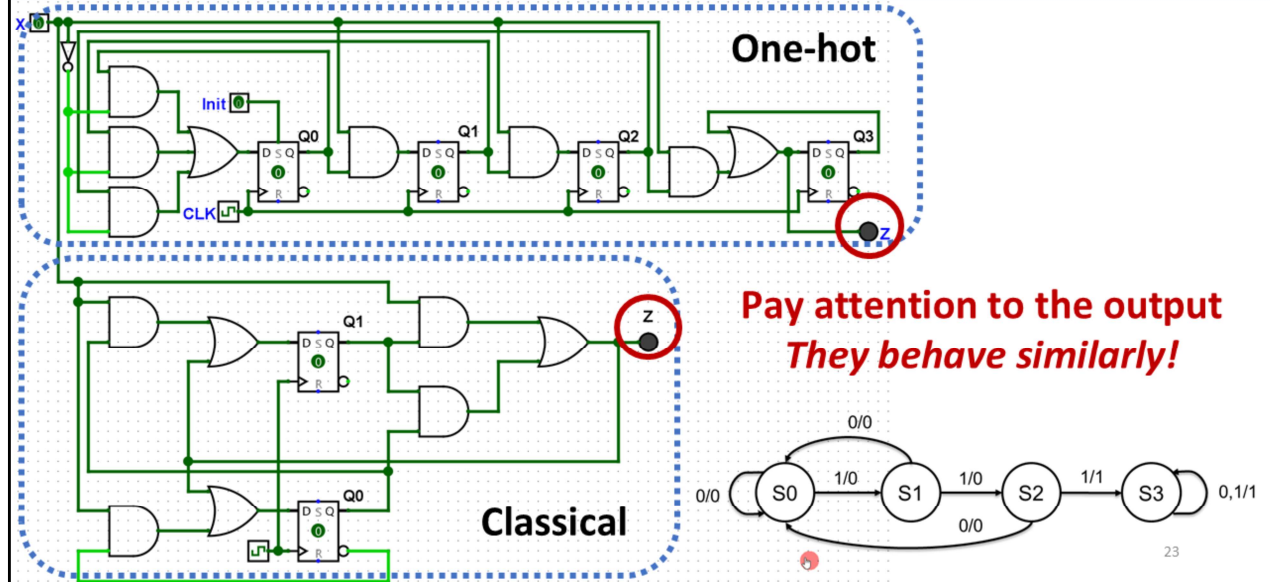
This is the 'one-hot way' (this lecture)

22

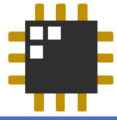
Now compare that to this slide, which shows the **one-hot encoding method**. We covered the entire process in just **5 slides**. That's because with one-hot encoding, we skip state assignment and Boolean minimization, and instead directly map each state to its own flip-flop. This makes the process **faster** and **easier to follow**, especially for smaller designs.



Putting it into perspective...



Here, we're comparing the **final circuits** produced by both methods. The classical method uses **fewer flip-flops**, but the logic is more **dense and intricate**. On the other hand, the one-hot version uses **more flip-flops**, but the logic is much **simpler and more transparent**. Also, with one-hot, it's very clear which state the system is in — because only one flip-flop is ON at a time.



Is one-hot encoding the best?

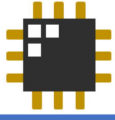
Advantages

- Straightforward next state logic
- Direct implementation from state diagram to flip-flop
 - Easy to debug (easier to track where you are in the state diagram)
 - Fast implementation (no need to fill out excitation tables and use Kmaps)
- Often leads to simpler (and therefore faster) output logic

Disadvantages?

24

One-hot encoding offers several advantages in sequential circuit design. The next-state logic is straightforward since each state is linked to a specific flip-flop, making state transitions clear and simple. It allows for direct implementation from the state diagram to flip-flops without the need for complex assignments or minimization steps. Debugging is easier because you can easily track the current state by checking which flip-flop is ON. The implementation process is faster as there's no need to fill out excitation tables or use Karnaugh maps. Additionally, the output logic tends to be simpler, often resulting in faster and more efficient circuits.



Is one-hot encoding the best?

Advantages

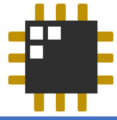
- Straightforward next state logic
- Direct implementation from state diagram to flip-flop
 - Easy to debug (easier to track where you are in the state diagram)
 - Fast implementation (no need to fill out excitation tables and use Kmaps)
- Often leads to simpler (and therefore faster) output logic

Disadvantage

- Needs more flip-flops: n states = n flip-flops (vs $\log_2 n$ for classical)

25

The main disadvantage of one-hot encoding is that it requires more flip-flops. For **n states**, you need **n flip-flops**, whereas the classical method only requires **$\log_2 n$ flip-flops**. This can lead to increased hardware usage, especially when dealing with a large number of states.

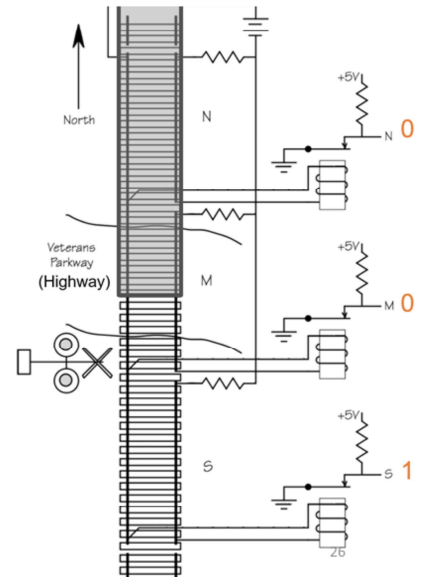


Practice

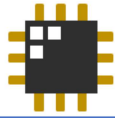
- The train progresses through the sections.
- If **train is long enough**, the three outputs go from NMS = 111 to 011 to 001 to 000 to 100 to 110 to 111 again.
- A short train could cause NMS = 111 to 011 to 001 to 101 to 100 to 110 to 111.

Draw the state diagram for a circuit that will activate the flashing highway signals. Call the signals F:

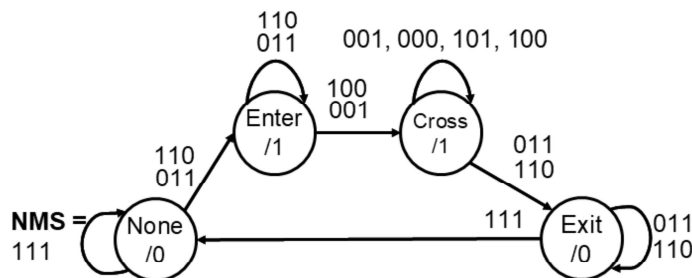
- When a train enters either N or S, the signal is to start flashing (F = 1)
- The signal is to stop when the train is out of the crossing itself (M).



Let's solve another example from our last lecture using one-hot encoding.



Classical state assignment (previous lecture)



Exercise:
Are the equations correct?
Does this work as intended?

Using DFFs and assigning:

None = 00, Enter = 01, Cross = 11, Exit = 10:

PS	Next State for input NMS								F
Q ₁ Q ₀	000	001	010	011	100	101	110	111	
00	xx	xx	xx	01	xx	xx	01	00	0
01	xx	11	xx	01	11	xx	01	xx	1
10	xx	xx	xx	10	xx	xx	10	xx	0
11	11	11	xx	10	11	11	10	xx	1

Using K-maps, we get the following expressions:

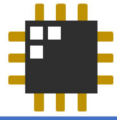
$$D_1 = N'Q_1 + M' + S'Q_1$$

$$D_0 = N'Q_1' + M' + S'Q_1'$$

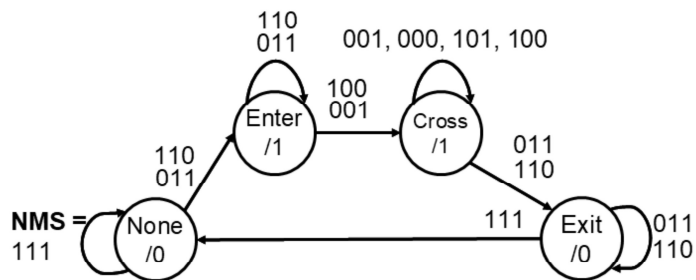
$$F = Q_0$$

Side comment: we didn't even show the Kmap procedure here...

In this slide, we see the **classical method** for solving the example. It's clear that the process involves several steps, including filling out the state table, excitation table, and using Karnaugh maps (K-maps) for simplification. **Just a side comment**, we didn't even include the K-map procedure here, but it's an essential part of the classical method. You can imagine that with each added step, the complexity increases, making it more cumbersome compared to the one-hot encoding approach.



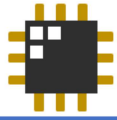
One-hot encoding practice



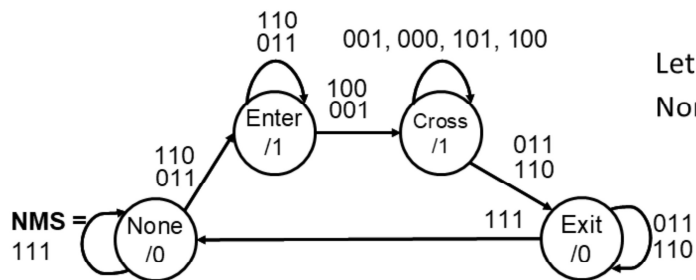
**Implement the circuit using
one-hot encoding instead**

28

Now, let's implement the circuit using a one-hot encoding instead.



One-hot encoding practice



Let

None = Q_0 Enter = Q_1 Cross = Q_2 Exit = Q_3

Next state equations:

$D_0 =$

$D_1 =$

$D_2 =$

$D_3 =$

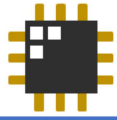
Output equation:

$F =$

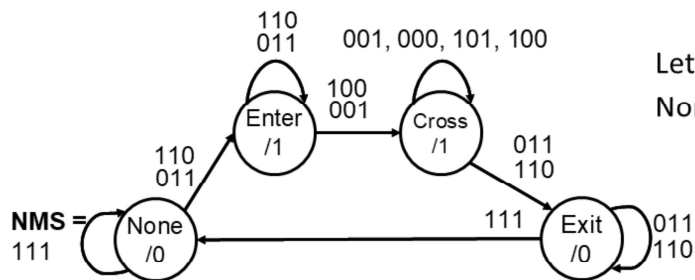
29

Let's define the states as follows:

None = Q_0 , **Enter** = Q_1 , **Cross** = Q_2 , **Exit** = Q_3 .



One-hot encoding practice



Let

None = Q_0 Enter = Q_1 Cross = Q_2 Exit = Q_3

Next state equations:

$$D_0 = NMS(Q_0 + Q_3)$$

$$D_1 =$$

$$D_2 =$$

$$D_3 =$$

Output equation:

$$F =$$

30

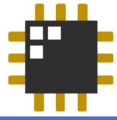
Now, let's start with the **Q0 (None)** state.

We transition to Q0 in two situations:

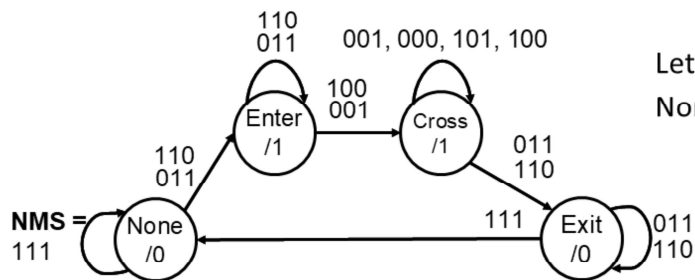
- When the current state is Q0 and the input is **NMS**
- When the current state is Q3 and the input is **NMS**

This gives us the equation:

$$D_0 = NMS(Q_0 + Q_3)$$



One-hot encoding practice



Let

None = Q_0 Enter = Q_1 Cross = Q_2 Exit = Q_3

Next state equations:

$$D_0 = NMS(Q_0 + Q_3)$$

$$D_1 = (NMS' + N'MS)(Q_0 + Q_1)$$

$$D_2 =$$

$$D_3 =$$

Output equation:

$$F =$$

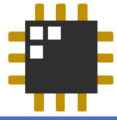
31

Now, let's move on to the **Q1 (Enter)** state. We transition to Q1 in the following situations:

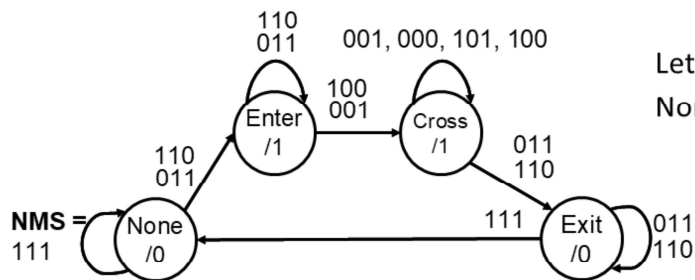
- When the current state is Q0 and the input is **NMS'** or **N'MS**
- When the current state is Q1 and the input is **NMS'** or **N'MS**

This gives us the equation:

$$D1 = (NMS' + N'MS)(Q0 + Q1)$$



One-hot encoding practice



Let

None = Q_0 Enter = Q_1 Cross = Q_2 Exit = Q_3

Next state equations:

$$D_0 = NMS(Q_0 + Q_3)$$

$$D_1 = (NMS' + N'MS)(Q_0 + Q_1)$$

$$D_2 = (NM'S' + N'M'S)Q_1 + (N'M'S + N'M'S' + NM'S + NM'S')Q_2$$

$$D_3 =$$

Output equation:

$$F =$$

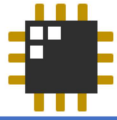
32

Now, let's move on to the **Q2 (Cross)** state. We transition to Q2 in the following situations:

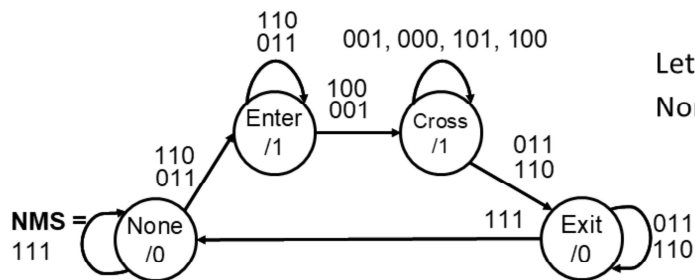
- When the current state is Q1 and the input is **NM'S'** or **N'M'S**
- When the current state is Q2 and the input is **N'M'S**, **N'M'S'**, **NM'S**, or **NM'S'**

This gives us the equation:

$$D_2 = (NM'S' + N'M'S)Q_1 + (N'M'S + N'M'S' + NM'S + NM'S')Q_2$$



One-hot encoding practice



Let

None = Q_0 Enter = Q_1 Cross = Q_2 Exit = Q_3

Next state equations:

$$D_0 = NMS(Q_0 + Q_3)$$

$$D_1 = (NMS' + N'MS)(Q_0 + Q_1)$$

$$D_2 = (NM'S' + N'M'S)Q_1 + (N'M'S + N'M'S' + NM'S + NM'S')Q_2$$

$$D_3 = (N'MS + NMS')(Q_2 + Q_3)$$

Output equation:

F =

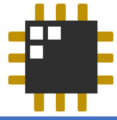
33

Finally, let's move on to the last state, **Q3 (Exit)**. We transition to Q3 in the following cases:

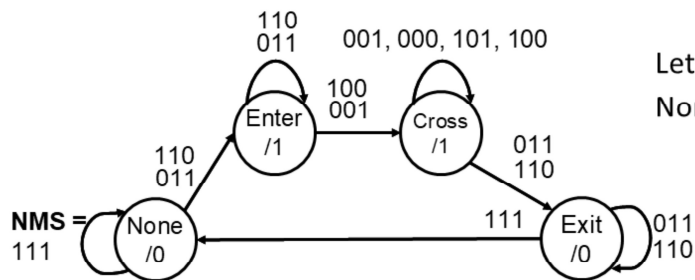
- When the current state is Q2 and the input is **N'MS** or **NMS'**
- When the current state is Q3 and the input is **N'MS** or **NMS'**

This leads to the equation:

$$D_3 = (N'MS + NMS')(Q_2 + Q_3)$$



One-hot encoding practice



Let

None = Q_0 Enter = Q_1 Cross = Q_2 Exit = Q_3

Next state equations:

$$D_0 = NMS(Q_0 + Q_3)$$

$$D_1 = (NMS' + N'MS)(Q_0 + Q_1)$$

$$D_2 = (NM'S' + N'M'S)Q_1 + (N'M'S + N'M'S' + NM'S + NM'S')Q_2$$

$$D_3 = (N'MS + NMS')(Q_2 + Q_3)$$

Output equation:

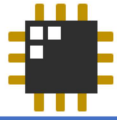
$$F = Q_1 + Q_2$$

34

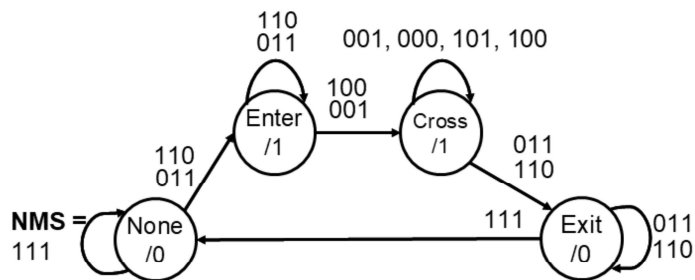
Lastly, let's look at the **output equation**. The output is **1** only when the system is in the **Q1 (Enter)** or **Q2 (Cross)** state.

This gives us the equation:

$$F = Q_1 + Q_2$$



One-hot encoding practice



Note: We could probably simplify the expressions even further by leveraging the fact that some input combinations are not possible (don't cares), but we won't go too much into that

Next state equations:

$$D_0 = NMS(Q_0 + Q_3)$$

$$D_1 = (NMS' + N'MS)(Q_0 + Q_1)$$

$$D_2 = (NM'S' + N'M'S)Q_1 + (N'M'S + N'M'S' + NM'S + NM'S')Q_2$$

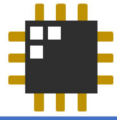
$$D_3 = (N'MS + NMS')(Q_2 + Q_3)$$

Output equation:

$$F = Q_1 + Q_2$$

35

Sometimes we can make our circuits even simpler by recognizing that certain input combinations never happen in practice. These are what we call *don't cares*. While we're not exploring that optimization technique just yet, it's good to be aware that such opportunities exist—especially as you work on more complex systems or aim to reduce gate count later on.



Conclusion

We now have two different techniques for synthesizing synchronous sequential circuits:

- **Classical method** (state diagram -> table -> Kmap -> circuit)
 - Optimizes for the number of flipflops in the circuit
- **One-hot encoding** (state diagram -> circuit)
 - Optimizes for synthesis speed

Never forget: As long as we drew the correct state diagram, *everything else is methodical!*

Translating a word problem into a state diagram correctly is critical

At this point, we've seen two distinct approaches to designing synchronous sequential circuits. The **classical method** is more efficient in terms of flip-flop usage, but it can be longer and more complex due to the need for tables and K-map simplification. On the other hand, **one-hot encoding** trades off flip-flop count for speed and simplicity—it gets you from the state diagram to the circuit much more directly.

But here's the key takeaway: *no matter which method you choose*, the design will only be as good as your **state diagram**. If you can translate the problem into a clear, accurate state diagram, the rest of the process becomes straightforward. So always take your time at that first step—it's the foundation for everything that follows.